SPEC-OPS · PROJECT RECAP

SPEC-OPS v2 — Project Recap

Status: onboarding live, attendance pipeline ready for tomorrow. **Date:** 2026-04-27. **Repo:** https://github.com/agencyaadu/specops-v2

What SPEC-OPS v2 is

A clean rebuild of the SPEC-OPS factory operations app. v1 (`spec-ops.best`) stays untouched in production while v2 ships incrementally on a new stack and a new schema.

The pivot vs v1:

- **Discord-first** — all field interactions happen via a bot (FREDDY). Operators, captains, chiefs, generals never touch a web form for daily work.
- **Webapp for setup + profile** — onboarding link, profile edit, upcoming admin/dashboard views. One URL, no app to install.
- **Single source of truth** — Supabase Postgres. Both Discord and the webapp write to the same tables. A background task mirrors selected tables to Google Sheets every 5 minutes.

Live URLs

- **Webapp:** https://bot-production-5dd6.up.railway.app
- **Discord bot: FREDDY** in the SPEC-OPS server
- **Database:** Supabase Postgres (project `specopsv2`)
- **Photo storage:** Supabase Storage bucket `specops-v2`
- **Sheets mirror:** Google Sheet `specopsv2mirror`
- **Hosting:** Railway (single service runs bot + web)

Stack

Layer	Choice
Bot framework	`discord.py` 2.x
Web framework	`aiohttp` (embedded in the bot process)
DB driver	`asyncpg`

Layer	Choice
Database	Supabase Postgres (transaction pooler)
File storage	Supabase Storage (photos, docs)
Sheets sync	`gspread` + Google service account
Auth (Discord)	Discord OAuth (native)
Auth (webapp)	Google OAuth + PAN, signed JWT cookie
Hosting	Railway, single service
Python	3.12

No Vercel, no FastAPI, no separate frontend service. One process.

Schema

Five tables, all UPPERCASE values where human-entered.

`people`

The roster. PAN is the primary identity.

Column	Notes
`pan`	TEXT PRIMARY KEY (e.g. `ABCDE1234F`)
`discord_id`, `discord_username`	Bot identity (NOT NULL, UNIQUE on discord_id)
`google_id`, `email`	Web login identity (UNIQUE on google_id)
`name`, `wa_number`	NOT NULL — captured at onboard
`dob`, `location`, `languages`, `hardest_problem`	Optional — filled via web profile
`headshot_url`, `intro_video_url`	Optional URLs
`bank_name`, `account_number`, `ifsc`, `upi_id`	Optional bank details

`factories`

Physical sites. One row per factory.

Column	Notes
`factory_id`	TEXT UNIQUE slug (e.g. `cse-1`)
`name`, `city`	UPPERCASE
`state`	`ACTIVE` / `INACTIVE`

`operations`

A working shift at a factory unit.

Column	Notes
`operation_id`	UNIQUE 4-segment ID: `CC-FF-Un-XX` all caps (city, factory, unit, shift)
`factory_id`, `shift`, `unit`, `city`	Components of the ID
`map_link`	Google Maps link
`poc1`, `poc2`	JSONB `{name, phone, role}`
`sales_team_name`	Internal team
`shift_start`, `shift_end`, `reporting_time`, `deployment_start`, `collection_start`, `report_submission_time`, `final_closing_time`	Timing fields, all `time` type

Examples: - `MU-AI-U1-NI` — Mumbai · Antariksh Infra · Unit 1 · Night - `JA-CS-U2-SA` — Jaipur · CSE · Unit 2 · Shift A - `PI-ST-U3-SA` — Pithampur · Shri Tirupati Balaji · Unit 3 · Shift A

A deterministic short-id generator (`bot/short_id.py`) builds these with collision-walking fallback.

`op_assignments`

Who works on which op, and at what rank.

Column	Notes
`operation_id`	FK → operations
`person_pan`, `person_discord_id`	NOT NULL — onboard required first

Column	Notes
`role`	`OPERATOR` / `CAPTAIN` / `CHIEF`
`state`	`ACTIVE` / `INACTIVE`
UNIQUE `(operation_id, person_pan, role)`	

`attendance`

One row per clock-in. Mutated for clock-out + validation.

Column	Notes
`at_id`	bigserial PK
`pp_pan`, `pp_discord_id`	The operator
`op_id`, `role`	Where & what
`clock_in_time`, `clock_out_time`	timestamptz
`photo_url`	Supabase Storage URL
`selected_validator_discord_id`	Picked at clock-in
`validator_discord_id`, `validation`, `validated_at`, `rejection_reason`	Set on Approve/Reject
`guild_id`, `validation_message_id`, `thread_id`	Discord plumbing

Uniqueness: one clock-in per (operator, op, IST date).

Discord (FREDDY) — slash commands

Command	Who can run	What it does
`/onboard`	anyone	In `#onboarding`, opens a modal: PAN + name + WhatsApp. After submit, shows a *Sign in with Google* button to link the Google account
`/clock-in`	onboarded operators	Op autocomplete + role + photo + validator autocomplete (only assigned validators show up). Posts an embed with Approve/Reject buttons in a private thread inside

Command	Who can run	What it does
		`#attendance-validation`, visible to the operator + selected validator only
`/clock-out`	onboarded operators	Closes the latest open tour with a confirm button
`/op-list`	anyone	Lists active operations
`/op-roster`	anyone	Shows everyone assigned to a given op
`/assign-chief`	FREDDY/GENERAL	Assigns a CHIEF to an op
`/assign-captain`	assigned CHIEF (+ above)	Assigns a CAPTAIN to an op
`/assign-operator`	assigned CAPTAIN (+ above)	Assigns an OPERATOR to an op
`/unassign`	rank-gated like assign	Removes someone from an op

Validation chain

When an operator clocks in, the bot:

1. Resolves the validator from the autocomplete pick.
2. Verifies they're assigned at the right rank for that op.
3. Creates a **private thread** in `#attendance-validation`, invites only the operator + the selected validator, and posts the embed with **Approve** + **Reject** buttons.
4. On Approve: status flips to `CONFIRMED`, embed colour changes, buttons disable.
5. On Reject: opens a modal asking for a reason; status flips to `REJECTED` with the reason captured.

Rank-up rule:

- OPERATOR clock-in → CAPTAIN validates
- CAPTAIN clock-in → CHIEF validates
- CHIEF clock-in → GENERAL/FREDDY validates

Self-validation is blocked.

Webapp routes

Hosted at https://bot-production-5dd6.up.railway.app — minimal pages, dark monochrome, no JS framework.

Route	What
`/`	Redirects to `/home` if signed in else `/login`
`/login`	Form: PAN + *Continue with Google* button. PAN+Google = two-factor login
`/auth/google/login`	Kicks off Google OAuth in login mode
`/google/callback`	Handles both link mode (from Discord button) and login mode
`/logout`	Clears session cookie
`/home`	Welcome card, stat tiles (active ops, total tours, confirmed), active op cards, View/Edit profile buttons
`/profile/<pan>`	Read-only profile view, gated to the signed-in user
`/profile/<pan>/edit`	Edit form for DOB, location, languages, hardest_problem, headshot URL, intro video URL, bank, account, IFSC, UPI
`POST /profile/<pan>`	Save profile edits
`/health`	Healthcheck

Session: signed JWT (HS256) in HttpOnly + Secure + SameSite=Lax cookie, 14-day TTL.

Sheets mirror

A background task in the bot process pushes selected tables to a Google Sheet every 5 minutes. Currently mirrors:

- `people`
- `attendance`

Each tab is a full-snapshot overwrite (simple, idempotent, slow tables).

Onboarding flow (end-to-end)

The journey for a new team member:

1. Run `/onboard` in `#onboarding`.
2. Tap **Open brief** → fill 3 fields (PAN, full name, WhatsApp).

3. After ✅ "you're on the SPEC-OPS roster", tap **Sign in with Google**.
4. Browser opens → pick the Google account → consent.
5. Lands on the *Onboarding complete* page → tap **Open my profile**.
6. On `/profile/<pan>`, tap **Edit profile**. Fill the rest of the profile (DOB, bank, headshot URL, intro video URL, etc.). Save.

After onboarding:

- A FREDDY/GENERAL assigns the person as a CHIEF on an op.
- That CHIEF assigns CAPTAINs.
- Those CAPTAINs assign OPERATORs.
- Operators run `/clock-in` → validator picks up Approve/Reject in the private thread.

Imported data so far

- **24 factories** + **29 operations** ETL'd from the v1 ops CSV. Cleanup pass: trim, slug, uppercase, dedupe; long IDs renamed to short `CC-FF-Un-XX` form (collision-protected).
- **31 chief/captain assignments** loaded from the v1 assignments CSV. Most skipped tonight because the chiefs haven't onboarded yet — re-run after they do.

What's done vs what's next

Done

- Schema (5 tables) live in production
- Discord bot online (FREDDY) with all 9 slash commands synced to the guild
- Onboarding flow (Discord modal → Google OAuth → web profile)
- Webapp (login, home, profile view + edit, logout)
- Validation: per-clock-in private thread + selected-validator gating
- Validator autocomplete from assignments
- Sheets mirror running every 5 minutes
- ETL scripts for ops + assignments CSVs

Next (tomorrow)

- **Webapp drag-drop assignments page** — replace the slash-command chain with a visual UI for adding/removing chiefs, captains, operators per op
- **Webapp factory + op management** — currently slash commands are removed, so creating new ops only happens via SQL/scripts until these pages land
- **Profile heatmap** — per active op, 7–14 day cell grid + past ops list (from the wireframes)

- **Per-op-per-role long-lived threads** — instead of per-clock-in private threads, one shared thread per (op, role-tier) for easier validator triage
- **Admin views** — today's attendance per op + cross-op pending validation queue
- **Validation reminder DM** — DM the validator if a request sits pending for > N hours

---

Key files in the repo

- `bot/main.py` — bot entry; boots discord client + web server + sheets mirror loop
- `bot/db.py` — schema + asyncpg pool
- `bot/short_id.py` — deterministic 2-char codes for the op_id
- `bot/web.py` — embedded aiohttp web server (login, profile, OAuth)
- `bot/commands/onboard.py` — modal + Google link button
- `bot/commands/attendance.py` — `/clock-in`, `/clock-out`, validator autocomplete
- `bot/commands/validate.py` — Approve/Reject buttons + role gating
- `bot/commands/admin.py` — `/op-list`, `/op-roster`, `/assign-*`, `/unassign`
- `bot/sheets_mirror.py` — Google Sheets push loop
- `bot/storage.py` — Supabase Storage upload helper
- `scripts/import_ops_csv.py` — one-shot ETL for ops CSV
- `scripts/import_assignments_csv.py` — one-shot ETL for assignments CSV (skips rows where the email isn't on the roster yet)
- `scripts/apply_short_ids.py` — one-shot rewrite of operation_ids to the short form
- `docs/schema.dbml` — full v2 schema (canonical reference; paste into dbdiagram.io)

Decision log (highlights from this build)

1. **Discord-first over webapp** for daily ops — no app installs, the field team is already on Discord all day.
2. **Single Railway service** instead of separating bot + web — one deploy, one log stream, half the infra to manage.
3. **Webapp embedded in the bot process** (aiohttp alongside discord.py) — no second backend, no separate auth surface.
4. **PAN as the human ID** — instead of an auto-generated `SO-NNNNN`, reuse the existing identity number people already carry.
5. **Drop op_password** — was on the design table for /clock-in, removed for the imported batch since rotation policy isn't decided yet. Can re-add when ops want it.
6. **Short op_ids** (`CC-FF-Un-XX` 4-segment, max ~16 chars) — long slugs were unscannable in Discord; the 2-char-per-segment rule with collision walk gives us readable but unique IDs.

7. **Slash commands per-guild sync** instead of global — global takes up to an hour to propagate; per-guild is instant and we're single-server for now.
8. **All-caps everywhere** for human-visible identifiers and team names — consistent visual signal for "this is a SPEC-OPS field".

This recap covers the build through 27 Apr 2026. Live deploy commit on Railway is on the `main` *branch of* `agencyaadu/specops-v2`*.*